

河狸云 AI 客户门户 - 内部技术方案

一、项目背景

1.1 业务背景

基于东威科技客户现场沟通反馈，我们发现：

1. 客户视角看板价值被高度认可

- 东威设备溢价20%+客户仍买单，核心是信任
- 客户希望实时了解交付进度，减少沟通成本
- 生益电子老板主动希望其他供应商也用河狸云

2. 现有看板的局限

- 客户需要多次点击、筛选才能找到信息
- 学习成本高，不够直观
- 只能查看，无法直接发起业务动作

1.2 项目目标

基于 Palantir AIP 方法论，构建 AI 客户门户：

- 让客户用自然语言对话了解交付进度
- 从"客户找信息"变成"信息找客户"
- 不只是查询，还能直接发起业务动作

1.3 核心理念

代码块

- 1 传统做法：数据 → 报表 → 人看 → 人决策
- 2 AI做法： 数据 → 本体对象 → AI理解 → 人机对话 → 行动

AI 不是独立的工具，而是嵌入到业务本体（Ontology）中的能力层

二、业务本体设计（Ontology）

2.1 什么是业务本体

业务本体是将业务概念抽象为**可被AI理解的对象模型**，包括：

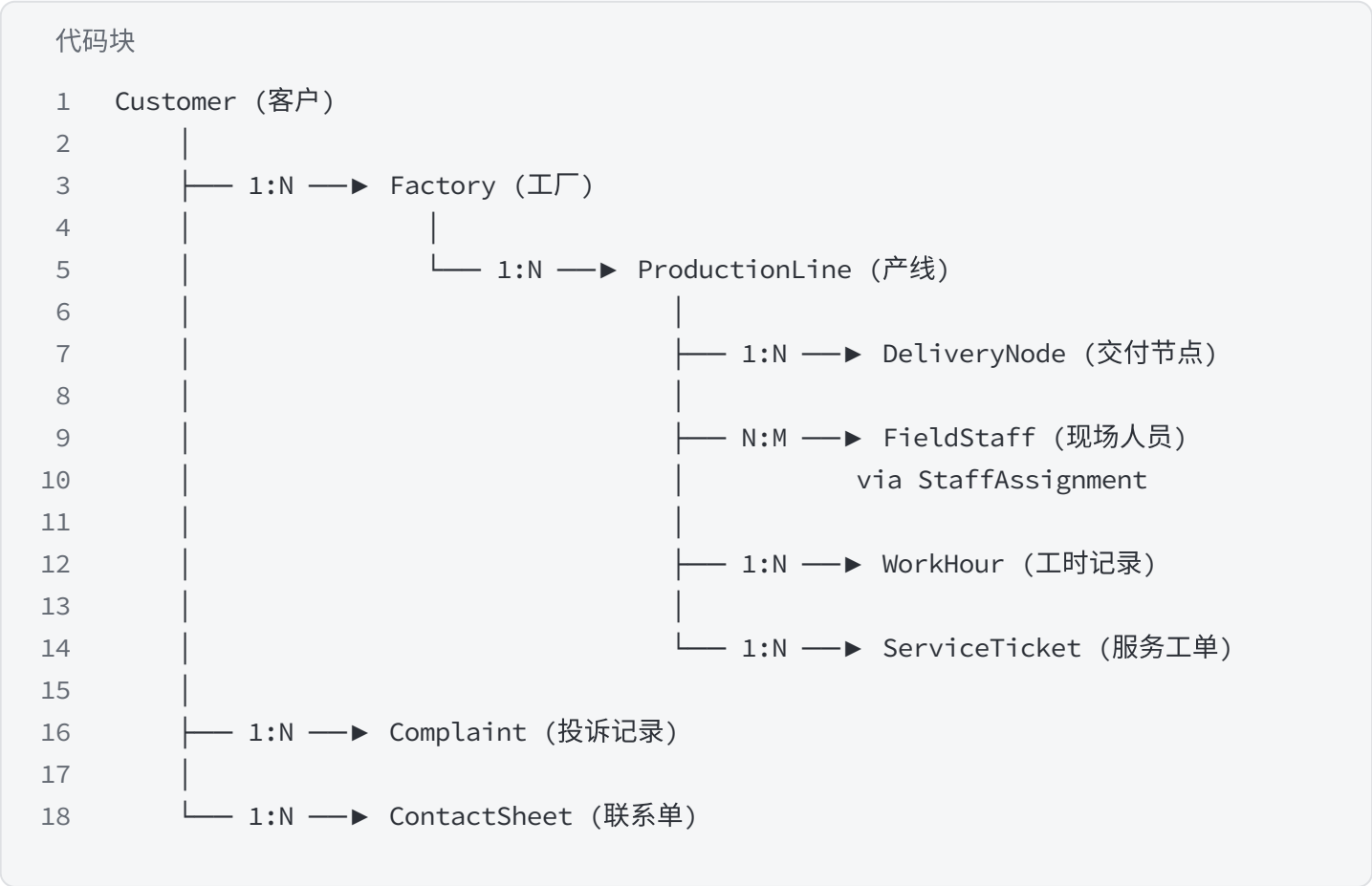
- 对象（Object）：业务实体
- 属性（Property）：对象的特征
- 关系（Relation）：对象之间的关联
- 规则（Rule）：业务逻辑约束

2.2 核心对象定义

对象	英文名	说明	主要属性
客户	Customer	终端客户企业	id, name, contact_person, phone, address
工厂	Factory	客户的工厂	id, customer_id, name, location
产线	ProductionLine	交付管理核心单元	id, factory_id, code, model, status, created_at
交付节点	DeliveryNode	进度跟踪最小单位	id, line_id, name, plan_start, plan_end, actual_start, actual_end, status, owner_id
现场人员	FieldStaff	施工/调试人员	id, name, department, role, phone
人员分配	StaffAssignment	人员与产线关系	id, staff_id, line_id, start_date, end_date
工时记录	WorkHour	工时统计	id, staff_id, line_id, date, hours, type(normal/overtime/holiday)
服务工单	ServiceTicket	服务报告等	id, line_id, type, status, handler_id, content, result, created_at
投诉记录	Complaint	客户投诉	id, customer_id, line_id, content, status,

			handler_id, result
联系单	ContactSheet	客户发起的联系	id, customer_id, line_id, type, content, status

2.3 对象关系图



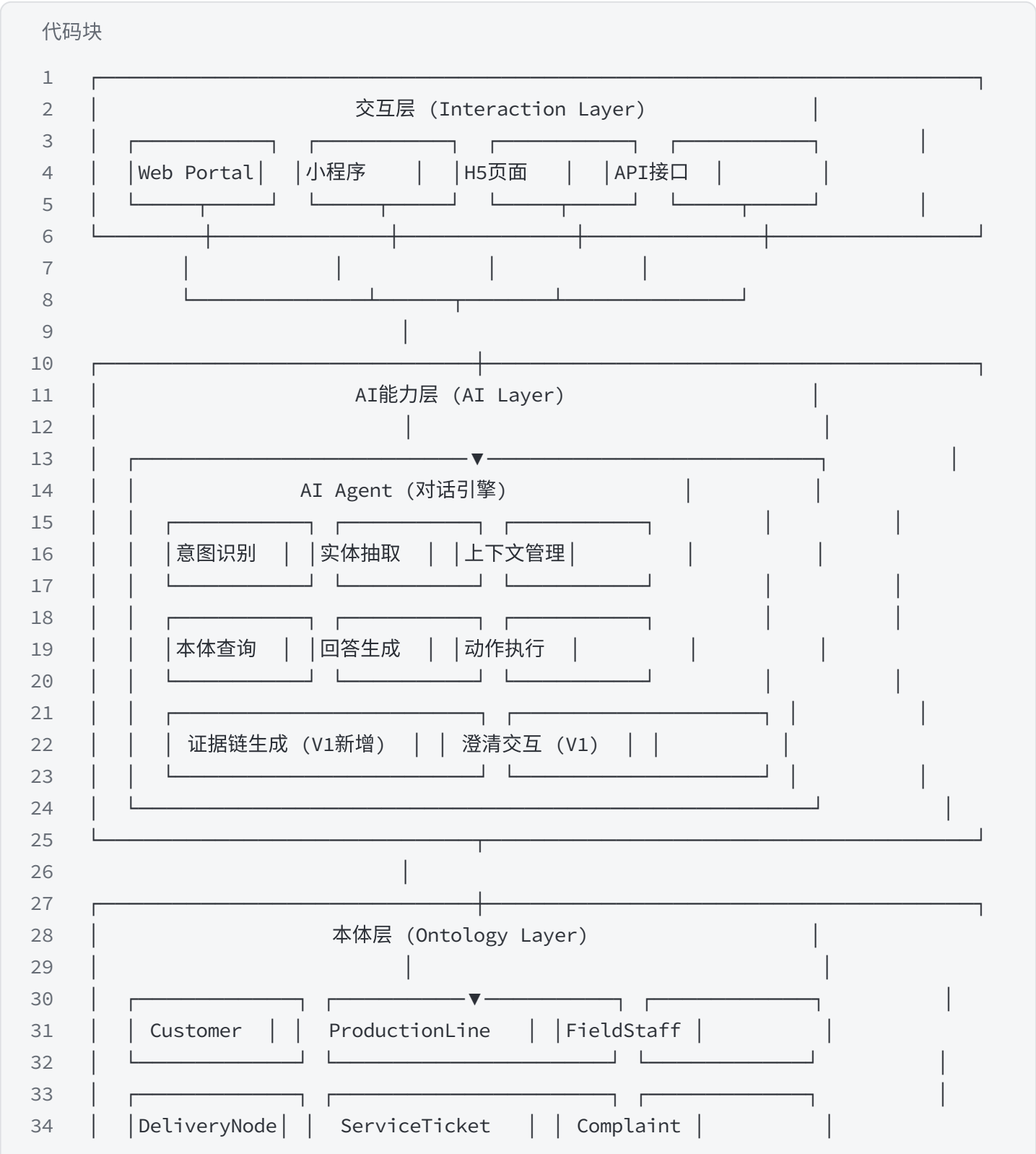
2.4 数据来源映射

本体对象	现有数据源	同步方式	备注
Customer	河狸云-客户表	实时	
Factory	河狸云-工厂表	实时	
ProductionLine	河狸云-产线表	实时	
DeliveryNode	河狸云-节点表	实时	需要包含计划和实际时间
FieldStaff	河狸云-员工表	准实时	
WorkHour	河狸云-工时表	每日同步	
ServiceTicket	河狸云-服务报告表	实时	

Complaint	河狸云-投诉表	实时	
ContactSheet	河狸云-联系单表	实时	

三、AI Agent 设计

3.1 整体架构




```
27     /** 关键字段摘要 */
28     private Map<String, Object> keyFields;
29     /** 人类可读的来源描述 */
30     private String description;
31 }
32 public enum EvidenceType {
33     RECORD,          // 单条记录
34     AGGREGATION,     // 聚合统计
35     SNAPSHOT         // 快照数据
36 }
```

3.2.2 各查询类型的证据要求

查询意图	必须携带的证据	示例描述
QUERY_PROGRESS	产线ID + 当前节点ID + 更新时间	产线 25B1339-G, 节点 #NODE042 更新于 14:27
QUERY_STAFF	人员分配记录ID + 生效日期	人员分配 #SA2024001, 01-15生效
QUERY_WORKHOUR	统计范围 + 记录条数	2026-01-01~01-31 共45条工时记录
QUERY_SERVICE	服务报告单号 + 报告日期	服务报告 #SR202601280001
QUERY_COMPLAINT	投诉单号 + 最新状态变更时间	投诉 #CP202601280001, 01-30 14:20更新
QUERY_ALERT	预警规则ID + 触发时间	逾期预警 #ALERT001, 01-28 09:00触发

3.2.3 回答结构（带证据链）

代码块

```
1  {
2      "reply": "东莞生益25B1339-G产线当前进度65%，处于调试阶段，现场3人。",
3      "reply_type": "progress_summary",
4      "structured_data": {
5          "line_code": "25B1339-G",
6          "progress": 65,
7          "current_stage": "调试",
8          "staff_count": 3
9      },
```

```
10  "evidence_chain": {
11    "query_time": "2026-02-04T14:32:00",
12    "freshness_note": "数据实时同步，最后更新于5分钟前",
13    "evidences": [
14      {
15        "type": "RECORD",
16        "source_type": "ProductionLine",
17        "record_id": "PL2025001339",
18        "update_time": "2026-02-04T14:27:00",
19        "updated_by": "系统自动",
20        "deep_link": "/portal/line/PL2025001339",
21        "key_fields": {"code": "25B1339-G", "status": "in_progress"},
22        "description": "产线 25B1339-G 基础信息"
23      },
24      {
25        "type": "AGGREGATION",
26        "source_type": "DeliveryNode",
27        "record_id": "agg_progress_PL2025001339",
28        "update_time": "2026-02-04T14:30:00",
29        "deep_link": "/portal/line/PL2025001339/nodes",
30        "key_fields": {"total": 65, "completed": 42, "rate": 0.65},
31        "description": "节点完成率 42/65"
32      }
33    ]
34  },
35  "quick_actions": [
36    {"label": "查看节点详情", "deep_link": "/portal/line/PL2025001339/nodes"},
37    {"label": "联系负责人", "action": "ACTION_CONTACT"}
38  ]
39 }
```

3.2.4 前端展示规范

代码块

1	
2	 AI助手
3	
4	东莞生益25B1339-G产线当前进度65%，处于调试阶段。
5	
6	 已完成42个节点，剩余23个
7	 现场3人：黄秋军(负责人)、张明、王磊
8	 无逾期节点
9	
10	
11	 数据来源 · 更新于5分钟前

12			• 产线信息 [查看→]		
13			• 节点统计 42/65 [查看节点→]		
14			• 人员分配 [查看分配→]		
15					
16					
17		[📄 查看节点] [👤 联系负责人] [📊 进度看板]			
18					

3.3 意图识别设计

3.2.1 意图分类

一期意图（MVP）：

意图ID	意图名称	说明	示例
QUERY_PROGRESS	进度查询	查询产线/节点进度	"我的产线进度怎么样了"
QUERY_STAFF	人员查询	查询现场人员情况	"现场有几个人"
QUERY_SERVICE	服务查询	查询服务报告等	"上次的服务报告"
QUERY_COMPLAINT	投诉查询	查询投诉处理进度	"我的投诉处理得怎样了"
QUERY_WORKHOUR	工时查询	查询工时统计	"这条产线花了多少工时"
QUERY_ALERT	预警查询	查询风险/预警	"有没有延期风险"
ACTION_COMPLAINT	发起投诉	创建投诉记录	"我要投诉"
ACTION_CONTACT	发起联系	创建联系单	"我要联系负责人"
CHITCHAT	闲聊	非业务对话	"你好"、"谢谢"

二期意图扩展（解释型/对比型/归因型）：

意图ID	意图名称	说明	示例
QUERY_DELAY_REASON	延期原因查询	查询节点延期的原因	"为什么这个节点延期了"
QUERY_VARIANCE	差异对比查询	对比不同时间/产线的差异	"工时为什么比上周多"
QUERY_CHANGELOG	变更记录查询	查询数据变更历史	"风险是谁确认的"
QUERY_TREND	趋势查询	查询进度/工时等趋势	"最近一个月进度怎么样"

扩展路径规划：

代码块

1

Path 1（更可控）：增加结构化意图

2

├── QUERY_DELAY_REASON → 关联"问题记录"表，抽取延期原因字段

3

├── QUERY_VARIANCE → 支持时间范围对比（本周 vs 上周）

4

└── QUERY_CHANGELOG → 关联操作日志表，返回变更人、变更时间

5

Path 2（更智能）：本体组合查询

6

├── 支持多API聚合（进度+工时+问题）

7

├── 仍要求输出结构化结果

8

└── 最后一步才做自然语言摘要

3.2.2 实体抽取

实体类型	说明	示例
LINE_CODE	产线编号	"25B1339-G"、"25B1340"
FACTORY_NAME	工厂名称	"东莞生益"、"4厂"
STAGE_NAME	阶段名称	"安装"、"调试"、"工艺"
TIME_RANGE	时间范围	"上周"、"这个月"、"最近"
PERSON_NAME	人员姓名	"黄秋军"
TICKET_ID	工单编号	"SP202601030001"

3.2.3 澄清交互产品化（关键设计）

设计原则：当意图不明确或缺少必要实体时，不要让客户"再打字"，而是提供可点击的选项按钮。

代码块

1

// ===== 澄清响应数据结构 =====

2

@Data

3

public class ClarificationResponse {

4

private String question; // 澄清问题

5

private ClarificationType type; // 澄清类型

6

private List<ClarificationOption> options; // 可选项（按钮）

7

private boolean allowFreeText; // 是否允许自由输入

8

}

9

public enum ClarificationType {

10

SELECT_LINE, // 选择产线

11

SELECT_STAGE, // 选择阶段

12

SELECT_TIME_RANGE, // 选择时间范围

```

13     SELECT_INTENT,          // 选择意图 (多个意图匹配时)
14     CONFIRM_ACTION          // 确认动作
15 }
16 @Data
17 public class ClarificationOption {
18     private String label;      // 显示文本
19     private String value;      // 值
20     private String icon;       // 图标 (可选)
21     private boolean isDefault; // 是否默认选中
22 }
23 // ===== 澄清场景示例 =====
24 // 场景1: 用户有多条产线, 未指定具体产线
25 {
26     "needs_clarification": true,
27     "clarification": {
28         "question": "您有3条进行中的产线, 请选择要查询的产线:",
29         "type": "SELECT_LINE",
30         "options": [
31             {"label": "东莞生益25B1339-G", "value": "25B1339-G", "icon": "🏭"},
32             {"label": "东莞生益25B1340-G", "value": "25B1340-G", "icon": "🏭"},
33             {"label": "深圳兴英25B1364-G", "value": "25B1364-G", "icon": "🏭"},
34             {"label": "查看全部产线", "value": "__ALL__", "icon": "📊"},
35         ],
36         "isDefault": true}
37     ],
38     "allowFreeText": false
39 }
40 // 场景2: 时间范围不明确
41 {
42     "needs_clarification": true,
43     "clarification": {
44         "question": "请选择要查询的时间范围:",
45         "type": "SELECT_TIME_RANGE",
46         "options": [
47             {"label": "今天", "value": "today", "icon": "📅"},
48             {"label": "本周", "value": "this_week", "icon": "📅", "isDefault": true},
49             {"label": "本月", "value": "this_month", "icon": "📅"},
50             {"label": "最近30天", "value": "last_30_days", "icon": "📅"}
51         ],
52         "allowFreeText": true
53     }
54 }
55 // 场景3: 意图模糊 (多个意图匹配)
56 {
57     "needs_clarification": true,
58     "clarification": {

```

```

58     "question": "请问您想：",
59     "type": "SELECT_INTENT",
60     "options": [
61         {"label": "查看投诉处理进度", "value": "QUERY_COMPLAINT", "icon":
        "🔍"},
62         {"label": "提交新的投诉", "value": "ACTION_COMPLAINT", "icon": "✍️"}
63     ],
64     "allowFreeText": true
65 }
66 }

```

前端交互规范：

代码块

```

1  |
2  | 🤖 您有3条进行中的产线，请选择要查询的产线： |
3  |
4  |
5  | | 🏭 东莞生益 | | 🏭 东莞生益 | |
6  | | 25B1339-G | | 25B1340-G | |
7  | |
8  | |
9  | | 🏭 深圳兴英 | | 📊 查看全部 | ← 默认 |
10 | | 25B1364-G | | 产线 | |
11 | |
12 |

```

交互流程：

1. 用户点击按钮 → 自动填充到输入框并发送
2. 后端收到带有明确实体的消息 → 正常处理
3. 上下文记住选择 → 后续对话自动关联该产线

3.3 本体查询引擎

3.3.1 查询模式

将自然语言转换为结构化查询：

代码块

```

1  # 示例：用户问 "我的产线进度怎么样了"
2  # Step 1: 意图识别
3  intent = "QUERY_PROGRESS"

```

```
4 # Step 2: 实体抽取 (无具体产线, 查全部)
5 entities = {}
6 # Step 3: 权限过滤
7 customer_id = session.customer_id
8 # Step 4: 构建查询
9 query = {
10     "object": "ProductionLine",
11     "filter": {
12         "customer_id": customer_id,
13         "status": ["in_progress", "pending"]
14     },
15     "include": ["delivery_nodes", "current_stage"],
16     "aggregate": ["progress_percentage", "overdue_count"]
17 }
18 # Step 5: 执行查询
19 result = ontology.query(query)
```

3.3.2 常用查询模板

意图	查询对象	关联对象	聚合计算
进度查询	ProductionLine	DeliveryNode	完成率、逾期数
人员查询	StaffAssignment	FieldStaff	人数、部门分布
服务查询	ServiceTicket	-	数量、状态分布
工时查询	WorkHour	FieldStaff	总工时、分类统计
预警查询	DeliveryNode	ProductionLine	逾期数、预警数

3.4 回答生成

3.4.1 回答模板

代码块

```
1 # 进度查询回答模板
2 PROGRESS_TEMPLATE = """
3 您好! {line_code}产线当前状态:
4 📊 整体进度: {current_stage}阶段, 完成{progress}%
5 📅 当前节点: {current_node}
6 👷 现场人员: {staff_count}人 ({department})
7 ⚠️ 预警: {alert_info}
8 {follow_up_question}
9 """
```

```

10 # 示例输出
11 ""
12 您好！东莞生益25B1339-G产线当前状态：
13 📊 整体进度：调试阶段，完成65%
14 📅 当前节点：槽体连接、固定
15 👤 现场人员：3人（调试部）
16 ⚠️ 预警：无逾期节点
17 需要查看详细节点或联系现场吗？
18 ""

```

3.4.2 证据链与数据溯源（关键设计）

设计原则：每条回答必须携带可验证的数据来源，让客户从"相信AI"变成"相信数据"。

代码块

```

1  // ===== 证据链数据结构 =====
2  @Data
3  public class EvidenceChain {
4      private String dataSource;           // 数据来源描述
5      private List<EvidenceItem> items;    // 证据项列表
6  }
7  @Data
8  public class EvidenceItem {
9      private String recordId;             // 记录ID/单据号
10     private String recordType;           // 记录类型（节点/工时/投诉等）
11     private String fieldName;            // 字段名称
12     private Object fieldValue;           // 字段值
13     private LocalDateTime updateTime;    // 数据更新时间
14     private String updatedBy;            // 更新人
15     private String jumpUrl;              // 跳转到原始记录的URL
16 }
17 // ===== 回答结构（带证据链） =====
18 @Data
19 public class AnswerWithEvidence {
20     private String answer;                // 自然语言回答
21     private EvidenceChain evidence;       // 证据链
22     private List<QuickAction> actions;    // 快捷操作（含跳转）
23 }
24 // ===== 进度查询回答示例 =====
25 {
26     "answer": "东莞生益25B1339-G产线当前处于调试阶段，整体完成65%，当前节点「槽体连接、固定」，现场3人。",
27     "evidence": {
28         "dataSource": "产线进度表 · 25B1339-G · 更新于2分钟前",
29         "items": [
30             {

```

```

31         "recordId": "NODE-20260203-0042",
32         "recordType": "delivery_node",
33         "fieldName": "当前节点",
34         "fieldValue": "槽体连接、固定",
35         "updateTime": "2026-02-03T14:30:00",
36         "updatedBy": "黄秋军",
37         "jumpUrl": "/portal/line/25B1339-G/node/42"
38     },
39     {
40         "recordId": "LINE-25B1339-G",
41         "recordType": "production_line",
42         "fieldName": "整体进度",
43         "fieldValue": "65%",
44         "updateTime": "2026-02-03T14:30:00",
45         "jumpUrl": "/portal/line/25B1339-G"
46     }
47 ]
48 },
49 "actions": [
50     {"label": "📄 查看节点详情", "action": "jump", "url":
"/portal/line/25B1339-G/node/42"},
51     {"label": "📊 查看进度看板", "action": "jump", "url":
"/portal/line/25B1339-G"},
52     {"label": "👤 联系负责人", "action": "intent", "intent":
"ACTION_CONTACT"}
53 ]
54 }

```

前端展示规范：

代码块

```

1  |
2  | 🤖 东莞生益25B1339-G产线当前处于调试阶段，          |
3  | 整体完成65%，当前节点「槽体连接、固定」...          |
4  |
5  | 📎 数据来源：产线进度表 · 更新于2分钟前              |
6  | └─ 当前节点：槽体连接、固定 → [查看详情]              |
7  | └─ 整体进度：65% → [查看看板]                          |
8  |
9  | [📄 查看节点详情] [📊 进度看板] [👤 联系负责人]      |
10 |

```

3.4.3 多轮对话上下文

上下文管理

```
1 class ConversationContext:
2     def __init__(self, customer_id):
3         self.customer_id = customer_id
4         self.current_line = None # 当前讨论的产线
5         self.current_topic = None # 当前话题
6         self.history = [] # 对话历史
7     def update(self, intent, entities, response):
8         # 更新当前上下文
9         if entities.get("line_code"):
10             self.current_line = entities["line_code"]
11         self.current_topic = intent
12         self.history.append({
13             "intent": intent,
14             "entities": entities,
15             "response": response
16         })
17
```

3.5 动作执行

3.5.1 动作流程

代码块

1 用户意图 → 参数收集 → 确认 → 执行 → 反馈

3.5.2 动作定义

代码块

```
1 // ===== 投诉动作 =====
2 @Component
3 public class ComplaintAction implements ActionHandler {
4     private static final List<String> REQUIRED_PARAMS =
5         Arrays.asList("content");
6     private static final List<String> OPTIONAL_PARAMS =
7         Arrays.asList("line_code", "urgency");
8     @Autowired
9     private ComplaintService complaintService;
10    @Override
11    public ActionType getType() {
12        return ActionType.ACTION_COMPLAINT;
13    }
14    @Override
15
```

```

13     public ActionResponse process(ConversationContext context, Map<String,
Object> params) {
14         // 检查必填参数
15         List<String> missing = getMissingParams(params);
16         if (!missing.isEmpty()) {
17             return ActionResponse.needMoreInfo(buildCollectPrompt(missing,
context));
18         }
19         // 生成确认信息
20         if (!Boolean.TRUE.equals(params.get("confirmed"))) {
21             return ActionResponse.needConfirm(buildConfirmMessage(params));
22         }
23         // 执行创建
24         Complaint complaint = complaintService.create(
25             context.getCustomerId(),
26             (String) params.get("line_code"),
27             (String) params.get("content"),
28             (String) params.getDefault("urgency", "normal")
29         );
30         return ActionResponse.success(buildSuccessMessage(complaint));
31     }
32     private String buildCollectPrompt(List<String> missing,
ConversationContext context) {
33         if (missing.contains("content")) {
34             String lineInfo = context.getCurrentLineCode() != null
35                 ? " (产线: " + context.getCurrentLineCode() + ") "
36                 : "";
37             return "好的, 我来帮您提交投诉" + lineInfo + "。请问具体是什么问题? ";
38         }
39         return "请补充以下信息: " + String.join(", ", missing);
40     }
41     private String buildConfirmMessage(Map<String, Object> params) {
42         return String.format("""
43             收到, 我来确认一下您的投诉信息:
44             📍 产线: %s
45             📝 问题: %s
46             ⚡ 紧急程度: %s
47             确认提交吗?
48             """,
49             params.getDefault("line_code", "未指定"),
50             params.get("content"),
51             "urgent".equals(params.get("urgency")) ? "紧急" : "普通"
52         );
53     }
54     private String buildSuccessMessage(Complaint complaint) {
55         return String.format("""
56             ✅ 投诉已提交!

```



```

57         📄 投诉编号: %s
58         👤 处理人: %s
59         📅 预计处理时间: 24小时内
60         我们会尽快处理, 有进展会及时通知您。您也可以随时问我"投诉进度"来查看处理情
        况。
61         """,
62         complaint.getCode(),
63         complaint.getHandlerName() != null ? complaint.getHandlerName() :
        "待分配"
64     );
65     }
66 }
67 // ===== 联系单动作 =====
68 @Component
69 public class ContactAction implements ActionHandler {
70     private static final List<String> REQUIRED_PARAMS =
        Arrays.asList("content");
71     private static final List<String> OPTIONAL_PARAMS =
        Arrays.asList("line_code", "contact_type");
72     @Autowired
73     private ContactSheetService contactService;
74     @Override
75     public ActionType getType() {
76         return ActionType.ACTION_CONTACT;
77     }
78     @Override
79     public ActionResponse process(ConversationContext context, Map<String,
        Object> params) {
80         // 检查必填参数
81         List<String> missing = getMissingParams(params);
82         if (!missing.isEmpty()) {
83             return ActionResponse.needMoreInfo(buildCollectPrompt(missing));
84         }
85         // 生成确认信息
86         if (!Boolean.TRUE.equals(params.get("confirmed"))) {
87             return ActionResponse.needConfirm(buildConfirmMessage(params));
88         }
89         // 执行创建
90         ContactSheet contact = contactService.create(
91             context.getCustomerId(),
92             (String) params.get("line_code"),
93             (String) params.getDefault("contact_type", "咨询"),
94             (String) params.get("content")
95         );
96         return ActionResponse.success(buildSuccessMessage(contact));
97     }
98     private String buildCollectPrompt(List<String> missing) {

```

```

99         return "好的，请问您想咨询什么内容？我会帮您转达给相关负责人。";
100     }
101     private String buildConfirmMessage(Map<String, Object> params) {
102         return String.format("""
103             收到，我来确认一下：
104             📝 内容：%s
105             📍 相关产线：%s
106             确认发送吗？我会通知相关负责人尽快与您联系。
107             """,
108             params.get("content"),
109             params.getDefault("line_code", "无")
110         );
111     }
112     private String buildSuccessMessage(ContactSheet contact) {
113         return String.format("""
114             ✅ 已收到您的联系请求！
115             📋 编号：%s
116             👤 负责人：%s
117             ☎️ 电话：%s
118             %s会尽快与您联系。如果紧急，您也可以直接拨打上方电话。
119             """,
120             contact.getCode(),
121             contact.getReceiverName(),
122             contact.getReceiverPhone(),
123             contact.getReceiverName()
124         );
125     }
126 }

```

四、接口设计

4.1 对话接口

请求

代码块

```

1  POST /api/ai/chat
2  Content-Type: application/json
3  Authorization: Bearer {token}
4  {
5      "session_id": "sess_abc123",
6      "message": "我的产线进度怎么样了？"
7  }

```

响应

代码块

```
1  {
2      "code": 0,
3      "data": {
4          "reply": "您好! 东莞生益25B1339-G产线当前状态...",
5          "reply_type": "text",
6          "structured_data": {
7              "lines": [
8                  {
9                      "code": "25B1339-G",
10                     "factory": "东莞生益",
11                     "status": "in_progress",
12                     "current_stage": "调试",
13                     "progress": 65,
14                     "staff_count": 3
15                 }
16             ]
17         },
18         "suggested_actions": [
19             {"label": "查看详细进度", "action": "view_detail", "params":
20 {"line_code": "25B1339-G"}},
21             {"label": "联系现场负责人", "action": "contact", "params":
22 {"line_code": "25B1339-G"}}
23         ],
24         "context": {
25             "current_line": "25B1339-G",
26             "current_topic": "QUERY_PROGRESS"
27         }
28     }
29 }
```

4.2 动作执行接口

请求

代码块

```
1  POST /api/ai/action
2  Content-Type: application/json
3  Authorization: Bearer {token}
4  {
5      "session_id": "sess_abc123",
```

```
6     "action": "create_complaint",
7     "params": {
8         "line_code": "25B1339-G",
9         "content": "上料机械手需要增加旋转功能"
10    },
11    "confirmed": true
12 }
```

响应

代码块

```
1  {
2      "code": 0,
3      "data": {
4          "success": true,
5          "result": {
6              "complaint_id": "CP202602030001",
7              "status": "pending",
8              "handler": "待分配"
9          },
10         "reply": "投诉已提交，编号：CP202602030001，我们会尽快处理并反馈给您。"
11     }
12 }
```

4.3 联系单接口

请求

代码块

```
1  POST /api/ai/action
2  Content-Type: application/json
3  Authorization: Bearer {token}
4  {
5      "session_id": "sess_abc123",
6      "action": "create_contact",
7      "params": {
8          "line_code": "25B1339-G",
9          "contact_type": "咨询",
10         "content": "想了解一下后续调试计划安排"
11     },
12     "confirmed": true
13 }
```

响应

代码块

```
1  {
2      "code": 0,
3      "data": {
4          "success": true,
5          "result": {
6              "contact_id": "CT202602030001",
7              "receiver": "黄秋军",
8              "receiver_phone": "138****5678"
9          },
10         "reply": "已收到您的联系请求！负责人黄秋军会尽快与您联系。"
11     }
12 }
```

4.4 会话管理接口

创建会话

代码块

```
1  POST /api/ai/session
2  Authorization: Bearer {token}
3  {
4      "customer_id": "C2024001"
5  }
```

获取会话历史

代码块

```
1  GET /api/ai/session/{session_id}/history
2  Authorization: Bearer {token}
```

五、权限与安全

5.1 数据权限矩阵

数据类型	客户可见	备注
------	------	----

自己的产线信息	✓	
自己产线的节点进度	✓	
现场人员姓名、部门	✓	
现场人员联系方式	✓	仅负责人
工时统计（汇总）	✓	仅总数，不含明细
工时统计（明细）	✗	涉及成本
服务报告	✓	
自己的投诉记录	✓	
内部成本数据	✗	
其他客户数据	✗	

5.2 身份认证

方案选择（待确认）：

方案	优点	缺点
手机号+验证码	简单、通用	需要发送短信
账号密码	传统、可控	用户需记住密码
企业微信授权	无感、便捷	需客户使用企微
微信小程序授权	便捷	需开发小程序

5.3 安全措施

- 1. 请求频率限制：单用户每分钟最多60次请求
- 2. 敏感词过滤：AI回答前过滤敏感信息
- 3. 操作审计：记录所有查询和动作
- 4. 会话超时：30分钟无操作自动过期

六、实施计划

6.1 阶段划分

代码块

- 1Phase 1: 本体建模 (2-4周)
- 2└─ Week 1-2: 业务对象梳理、属性定义
- 3└─ Week 2-3: 数据源对接、同步机制
- 4└─ Week 3-4: 本体验证、调整优化
- 5Phase 2: AI Agent 开发 (4-6周)
- 6└─ Week 1-2: 意图识别模型
- 7└─ Week 2-3: 实体抽取、本体查询
- 8└─ Week 3-4: 回答生成、模板配置
- 9└─ Week 4-5: 动作执行开发
- 10└─ Week 5-6: 上下文管理、联调测试
- 11Phase 3: 客户端集成 (2-4周)
- 12└─ Week 1-2: Web门户开发
- 13└─ Week 2-3: 嵌入客户看板
- 14└─ Week 3-4: 移动端适配
- 15Phase 4: 测试上线 (2-3周)
- 16└─ Week 1: 内部测试
- 17└─ Week 2: 客户试点
- 18└─ Week 3: 正式上线

6.2 里程碑

里程碑	时间点	交付物
M1	第4周	本体模型、数据同步完成
M2	第10周	AI Agent 核心功能完成
M3	第14周	Web门户可用
M4	第17周	正式上线

6.3 资源需求

角色	人数	工作内容	技术要求
产品经理	1	需求细化、验收	-
Java后端开发	2	本体层、AI Agent、API	Spring Boot, MySQL, Redis
React前端开发	1	Web门户、嵌入集成	React, TypeScript
Flutter开发	1	移动端APP	Flutter, Dart

AI工程师	0.5	意图识别、大模型对接	NLP基础, LLM API
测试	1	功能测试、场景测试	-

七、风险与应对

风险	可能性	影响	应对措施
意图识别准确率不够	中	高	增加训练数据、人工兜底
数据同步延迟	低	中	实时同步+缓存策略
客户使用率低	中	高	优化体验、引导教育
安全问题	低	高	严格权限控制、审计日志

八、附录

8.1 意图识别训练数据示例

代码块

```
1  [
2      {"text": "我的产线进度怎么样了", "intent": "QUERY_PROGRESS", "entities": []},
3      {"text": "25B1339这条线到什么阶段了", "intent": "QUERY_PROGRESS", "entities":
4          [{"type": "LINE_CODE", "value": "25B1339"}]},
5      {"text": "现场有几个人在施工", "intent": "QUERY_STAFF", "entities": []},
6      {"text": "今天谁在负责调试", "intent": "QUERY_STAFF", "entities": [{"type":
7          "STAGE_NAME", "value": "调试"}]},
8      {"text": "我要投诉一个问题", "intent": "ACTION_COMPLAINT", "entities": []},
9      {"text": "我要申请工艺变更", "intent": "ACTION_CHANGE", "entities": [{"type":
10         "CHANGE_TYPE", "value": "工艺"}]}
11 ]
```

8.2 技术选型（基于现有技术栈）

现有技术栈：Java + MySQL + Redis + RabbitMQ + React + Flutter

组件	技术方案	说明
----	------	----

| 后端框架 | Spring Boot 3.x | 现有技术栈延续 |

| 数据存储 | MySQL 8.x | 本体对象存储，利用JSON字段增强灵活性 |

| 缓存 | Redis | 会话上下文、热点数据缓存 |

| 消息队列 | RabbitMQ | 异步任务、动作执行、通知推送 |

| 意图识别 | 大模型API + 本地规则 | 见下方详细说明 |

| Web前端 | React | 客户门户Web版 |

| 移动端 | Flutter | iOS/Android统一开发 |

| API网关 | Spring Cloud Gateway | 统一入口、限流、鉴权 |

8.3 Java 后端架构设计

8.3.1 模块划分

代码块

```
1  ai-customer-portal/  
2  |—— portal-api/                # API层 (Controller)  
3  |   |—— ChatController.java     # 对话接口  
4  |   |—— ActionController.java  # 动作执行接口  
5  |   |—— SessionController.java # 会话管理接口  
6  |  
7  |—— portal-service/            # 业务层  
8  |   |—— ai/  
9  |       |—— AIAgentService.java      # AI对话主服务  
10 |       |—— IntentRecognitionService.java # 意图识别  
11 |       |—— EntityExtractionService.java # 实体抽取  
12 |       |—— OntologyQueryService.java   # 本体查询  
13 |       |—— ResponseGeneratorService.java # 回答生成  
14 |       |—— ActionExecutorService.java  # 动作执行  
15 |  
16 |   |—— session/  
17 |       |—— SessionService.java         # 会话管理  
18 |       |—— ContextManager.java        # 上下文管理  
19 |  
20 |   |—— ontology/  
21 |       |—— CustomerOntologyService.java  
22 |       |—— ProductionLineOntologyService.java  
23 |       |—— ...  
24 |  
25 |—— portal-domain/             # 领域模型  
26 |   |—— entity/                 # 本体对象实体  
27 |   |—— dto/                   # 数据传输对象  
28 |   |—— enums/                 # 枚举 (意图、状态等)
```

```

29 |
30 |—— portal-infrastructure/          # 基础设施层
31 |   |—— llm/                        # 大模型集成
32 |   |   |—— LLMClient.java
33 |   |   |—— LLMConfig.java
34 |   |—— mq/                        # 消息队列
35 |   |—— cache/                    # Redis缓存
36 |
37 |—— portal-common/                # 公共模块
38 |   |—— exception/
39 |   |—— utils/

```

8.3.2 核心类设计

代码块

```

1  // ===== 意图枚举 =====
2  public enum IntentType {
3      QUERY_PROGRESS("进度查询"),
4      QUERY_STAFF("人员查询"),
5      QUERY_SERVICE("服务查询"),
6      QUERY_COMPLAINT("投诉查询"),
7      QUERY_WORKHOUR("工时查询"),
8      QUERY_ALERT("预警查询"),
9      ACTION_COMPLAINT("发起投诉"),
10     ACTION_CONTACT("发起联系"),
11     CHITCHAT("闲聊");
12 }
13 // ===== 会话上下文 =====
14 @Data
15 public class ConversationContext implements Serializable {
16     private String sessionId;
17     private Long customerId;
18     private String currentLineCode;    // 当前讨论的产线
19     private IntentType currentTopic;   // 当前话题
20     private Map<String, Object> slots;  // 槽位 (收集的参数)
21     private List<Message> history;     // 对话历史
22     private LocalDateTime lastActiveTime;
23 }
24 // ===== AI Agent 主服务 =====
25 @Service
26 @Slf4j
27 public class AIAgentService {
28     @Autowired
29     private IntentRecognitionService intentService;
30     @Autowired

```

```

31     private EntityExtractionService entityService;
32     @Autowired
33     private OntologyQueryService ontologyService;
34     @Autowired
35     private ResponseGeneratorService responseService;
36     @Autowired
37     private ActionExecutorService actionService;
38     @Autowired
39     private SessionService sessionService;
40     public ChatResponse chat(ChatRequest request) {
41         // 1. 获取/创建会话上下文
42         ConversationContext context =
sessionService.getOrCreateContext(request.getSessionId());
43         // 2. 意图识别
44         IntentResult intent = intentService.recognize(request.getMessage(),
context);
45         // 3. 实体抽取
46         List<Entity> entities = entityService.extract(request.getMessage(),
intent.getType());
47         // 4. 更新上下文
48         updateContext(context, intent, entities);
49         // 5. 根据意图类型处理
50         ChatResponse response;
51         if (intent.getType().isQuery()) {
52             // 查询类: 执行本体查询 + 生成回答
53             OntologyQueryResult queryResult = ontologyService.query(intent,
entities, context);
54             response = responseService.generate(intent, queryResult, context);
55         } else if (intent.getType().isAction()) {
56             // 动作类: 检查参数 + 确认 + 执行
57             response = actionService.process(intent, entities, context);
58         } else {
59             // 闲聊类
60             response = responseService.generateChitchat(request.getMessage());
61         }
62         // 6. 保存上下文
63         sessionService.saveContext(context);
64         return response;
65     }
66 }
67 // ===== 意图识别服务 =====
68 @Service
69 public class IntentRecognitionService {
70     @Autowired
71     private LLMClient llmClient;
72     @Autowired
73     private IntentRuleEngine ruleEngine;

```

```

74     public IntentResult recognize(String message, ConversationContext context)
75     {
76         // 策略1: 规则引擎快速匹配 (高频、明确的意图)
77         IntentResult ruleResult = ruleEngine.match(message);
78         if (ruleResult != null && ruleResult.getConfidence() > 0.9) {
79             return ruleResult;
80         }
81         // 策略2: 大模型识别 (复杂、模糊的意图)
82         String prompt = buildIntentPrompt(message, context);
83         String llmResponse = llmClient.complete(prompt);
84         return parseIntentFromLLM(llmResponse);
85     }
86     private String buildIntentPrompt(String message, ConversationContext
context) {
87         return String.format("""
88             你是一个意图识别助手。请分析用户输入，返回意图类型。
89             可选意图：
90             - QUERY_PROGRESS: 查询产线进度
91             - QUERY_STAFF: 查询现场人员
92             - QUERY_SERVICE: 查询服务报告
93             - QUERY_COMPLAINT: 查询投诉进度
94             - ACTION_COMPLAINT: 发起投诉
95             - ACTION_CHANGE: 申请变更
96             - CHITCHAT: 闲聊
97             当前上下文: 正在讨论产线 %s
98             用户输入: %s
99             请返回JSON格式: {"intent": "xxx", "confidence": 0.95}
100             """,
101             context.getCurrentLineCode(),
102             message
103         );
104     }
105     // ===== 本体查询服务 =====
106     @Service
107     public class OntologyQueryService {
108         @Autowired
109         private ProductionLineRepository lineRepository;
110         @Autowired
111         private DeliveryNodeRepository nodeRepository;
112         @Autowired
113         private FieldStaffRepository staffRepository;
114         // ... 其他Repository
115         public OntologyQueryResult query(IntentResult intent, List<Entity>
entities,
116             ConversationContext context) {
117             // 根据意图类型路由到不同的查询方法

```

```

118         return switch (intent.getType()) {
119             case QUERY_PROGRESS -> queryProgress(entities, context);
120             case QUERY_STAFF -> queryStaff(entities, context);
121             case QUERY_SERVICE -> queryService(entities, context);
122             case QUERY_COMPLAINT -> queryComplaint(entities, context);
123             case QUERY_WORKHOUR -> queryWorkHour(entities, context);
124             case QUERY_ALERT -> queryAlert(entities, context);
125             default -> OntologyQueryResult.empty();
126         };
127     }
128     private OntologyQueryResult queryProgress(List<Entity> entities,
129                                             ConversationContext context) {
130         // 确定查询的产线
131         String lineCode = extractLineCode(entities, context);
132         Long customerId = context.getCustomerId();
133         // 查询产线信息
134         List<ProductionLine> lines;
135         if (lineCode != null) {
136             lines = lineRepository.findByCustomerIdAndCode(customerId,
137 lineCode);
138         } else {
139             lines = lineRepository.findByCustomerIdAndStatusIn(
140                 customerId, Arrays.asList("in_progress", "pending")
141             );
142         }
143         // 查询关联的节点信息
144         for (ProductionLine line : lines) {
145             List<DeliveryNode> nodes =
146 nodeRepository.findByLineId(line.getId());
147             line.setNodes(nodes);
148             // 计算进度统计
149             ProgressStats stats = calculateProgress(nodes);
150             line.setProgressStats(stats);
151         }
152         return OntologyQueryResult.builder()
153             .type(QueryResultType.PROGRESS)
154             .data(lines)
155             .build();
156     }
157 }

```

8.3.3 Redis 缓存设计

代码块

```
1 // 缓存Key设计
```

```

2  public class CacheKeys {
3      // 会话上下文: portal:session:{sessionId}, TTL 30分钟
4      public static String sessionContext(String sessionId) {
5          return "portal:session:" + sessionId;
6      }
7      // 客户产线列表: portal:customer:{customerId}:lines, TTL 5分钟
8      public static String customerLines(Long customerId) {
9          return "portal:customer:" + customerId + ":lines";
10     }
11     // 产线进度: portal:line:{lineCode}:progress, TTL 1分钟
12     public static String lineProgress(String lineCode) {
13         return "portal:line:" + lineCode + ":progress";
14     }
15 }
16 // 会话管理服务
17 @Service
18 public class SessionService {
19     @Autowired
20     private RedisTemplate<String, Object> redisTemplate;
21     private static final long SESSION_TTL_MINUTES = 30;
22     public ConversationContext getOrCreateContext(String sessionId) {
23         String key = CacheKeys.sessionContext(sessionId);
24         ConversationContext context = (ConversationContext)
redisTemplate.opsForValue().get(key);
25         if (context == null) {
26             context = new ConversationContext();
27             context.setSessionId(sessionId);
28             context.setHistory(new ArrayList<>());
29             context.setSlots(new HashMap<>());
30         }
31         context.setLastActiveTime(LocalDateTime.now());
32         return context;
33     }
34     public void saveContext(ConversationContext context) {
35         String key = CacheKeys.sessionContext(context.getSessionId());
36         redisTemplate.opsForValue().set(key, context, SESSION_TTL_MINUTES,
TimeUnit.MINUTES);
37     }
38 }

```

8.3.4 RabbitMQ 消息设计

代码块

```

1  // 消息队列配置
2  @Configuration

```

```
3 public class RabbitMQConfig {
4     // 动作执行队列
5     public static final String ACTION_QUEUE = "portal.action.execute";
6     // 通知推送队列
7     public static final String NOTIFICATION_QUEUE = "portal.notification.push";
8     @Bean
9     public Queue actionQueue() {
10         return new Queue(ACTION_QUEUE, true);
11     }
12     @Bean
13     public Queue notificationQueue() {
14         return new Queue(NOTIFICATION_QUEUE, true);
15     }
16 }
17 // 动作执行消息
18 @Data
19 public class ActionMessage implements Serializable {
20     private String actionId;
21     private String actionType; // complaint, contact
22     private Long customerId;
23     private String lineCode;
24     private Map<String, Object> params;
25     private LocalDateTime createTime;
26 }
27 // 动作执行消费者
28 @Component
29 @RabbitListener(queues = RabbitMQConfig.ACTION_QUEUE)
30 public class ActionExecuteConsumer {
31     @Autowired
32     private ComplaintService complaintService;
33     @Autowired
34     private ContactSheetService contactService;
35     @RabbitHandler
36     public void handle(ActionMessage message) {
37         log.info("Processing action: {}", message.getActionId());
38         try {
39             switch (message.getActionType()) {
40                 case "complaint" -> complaintService.create(message);
41                 case "contact" -> contactService.create(message);
42             }
43         } catch (Exception e) {
44             log.error("Action execute failed: {}", message.getActionId(), e);
45             // 发送失败通知
46         }
47     }
48 }
```

8.4 数据库设计

8.4.1 新增表（AI客户门户专用）

代码块

```
1  -- 会话记录表（审计用，Redis存实时上下文）
2  CREATE TABLE ai_chat_session (
3      id BIGINT PRIMARY KEY AUTO_INCREMENT,
4      session_id VARCHAR(64) NOT NULL UNIQUE,
5      customer_id BIGINT NOT NULL,
6      start_time DATETIME NOT NULL,
7      end_time DATETIME,
8      message_count INT DEFAULT 0,
9      created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
10     INDEX idx_customer_id (customer_id),
11     INDEX idx_start_time (start_time)
12 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
13 -- 对话消息记录表
14 CREATE TABLE ai_chat_message (
15     id BIGINT PRIMARY KEY AUTO_INCREMENT,
16     session_id VARCHAR(64) NOT NULL,
17     role ENUM('user', 'assistant') NOT NULL,
18     content TEXT NOT NULL,
19     intent VARCHAR(32),
20     entities JSON,
21     created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
22     INDEX idx_session_id (session_id),
23     INDEX idx_created_at (created_at)
24 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
25 -- 动作执行记录表
26 CREATE TABLE ai_action_log (
27     id BIGINT PRIMARY KEY AUTO_INCREMENT,
28     session_id VARCHAR(64) NOT NULL,
29     customer_id BIGINT NOT NULL,
30     action_type VARCHAR(32) NOT NULL,
31     action_params JSON,
32     status ENUM('pending', 'success', 'failed') DEFAULT 'pending',
33     result JSON,
34     error_message TEXT,
35     created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
36     executed_at DATETIME,
37     INDEX idx_customer_id (customer_id),
38     INDEX idx_action_type (action_type),
39     INDEX idx_status (status)
40 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```


8.5 React 前端设计

8.5.1 组件结构

代码块

```
1  src/
2  |— components/
3  |   └─ AIChat/
4  |       └─ index.tsx          # 主组件
5  |       └─ ChatWindow.tsx     # 对话窗口
6  |       └─ MessageList.tsx    # 消息列表
7  |       └─ MessageItem.tsx    # 单条消息
8  |       └─ InputBox.tsx       # 输入框
9  |       └─ ActionButton.tsx   # 快捷动作按钮
10 |       └─ StructuredData.tsx  # 结构化数据展示
11 |       └─ styles.module.scss  # 样式
12 |
13 |— hooks/
14 |   └─ useAIChat.ts           # 对话Hook
15 |
16 |— services/
17 |   └─ aiChatService.ts       # API调用
18 |
19 |— types/
20 |   └─ aiChat.ts              # 类型定义
```

8.5.2 核心组件示例

代码块

```
1  // useAIChat.ts - 对话Hook
2  import { useState, useCallback } from 'react';
3  import { aiChatService } from '@services/aiChatService';
4  export const useAIChat = (customerId: string) => {
5    const [sessionId, setSessionId] = useState<string | null>(null);
6    const [messages, setMessages] = useState<Message[]>([]);
7    const [loading, setLoading] = useState(false);
8    const sendMessage = useCallback(async (content: string) => {
9      // 添加用户消息
10     const userMsg: Message = { role: 'user', content, timestamp: Date.now() };
11     setMessages(prev => [...prev, userMsg]);
12     setLoading(true);
13     try {
14       const response = await aiChatService.chat({
15         sessionId,
16         message: content,
```

```
17     });
18     // 更新sessionId
19     if (!sessionId) {
20         setSessionId(response.sessionId);
21     }
22     // 添加AI回复
23     const aiMsg: Message = {
24         role: 'assistant',
25         content: response.reply,
26         structuredData: response.structuredData,
27         suggestedActions: response.suggestedActions,
28         timestamp: Date.now(),
29     };
30     setMessages(prev => [...prev, aiMsg]);
31 } catch (error) {
32     // 错误处理
33     const errorMsg: Message = {
34         role: 'assistant',
35         content: '抱歉, 出现了一些问题, 请稍后再试。',
36         timestamp: Date.now(),
37     };
38     setMessages(prev => [...prev, errorMsg]);
39 } finally {
40     setLoading(false);
41 }
42 }, [sessionId]);
43 const executeAction = useCallback(async (action: string, params: any) => {
44     setLoading(true);
45     try {
46         const response = await aiChatService.executeAction({
47             sessionId,
48             action,
49             params,
50             confirmed: true,
51         });
52         // 添加执行结果消息
53         const resultMsg: Message = {
54             role: 'assistant',
55             content: response.reply,
56             timestamp: Date.now(),
57         };
58         setMessages(prev => [...prev, resultMsg]);
59     } finally {
60         setLoading(false);
61     }
62 }, [sessionId]);
63 return { messages, loading, sendMessage, executeAction };
```

```

64   };
65   // ChatWindow.tsx - 对话窗口组件
66   import React, { useState } from 'react';
67   import { useAIChat } from '@/hooks/useAIChat';
68   import MessageList from './MessageList';
69   import InputBox from './InputBox';
70   import styles from './styles.module.scss';
71   interface Props {
72     customerId: string;
73     visible: boolean;
74     onClose: () => void;
75   }
76   const ChatWindow: React.FC<Props> = ({ customerId, visible, onClose }) => {
77     const { messages, loading, sendMessage, executeAction } =
78       useAIChat(customerId);
79     if (!visible) return null;
80     return (
81       <div className={styles.chatWindow}>
82         <div className={styles.header}>
83           <div className={styles.title}>
84             <span className={styles.avatar}>🤖</span>
85             <span>河狸云 AI 助手</span>
86           </div>
87           <button className={styles.closeBtn} onClick={onClose}>✕</button>
88         </div>
89         <MessageList
90           messages={messages}
91           onActionClick={executeAction}
92         />
93         <InputBox
94           onSend={sendMessage}
95           disabled={loading}
96           placeholder="输入您的问题..."
97         />
98       </div>
99     );
100   };
101   export default ChatWindow;

```

8.6 Flutter 移动端设计

8.6.1 目录结构

代码块

1 lib/

```

2  |── features/
3  |   └── ai_chat/
4  |       ├── data/
5  |           ├── models/
6  |               ├── message_model.dart
7  |               └── chat_response_model.dart
8  |           └── repositories/
9  |               └── ai_chat_repository.dart
10 |       ├── domain/
11 |           ├── entities/
12 |               └── message.dart
13 |       └── presentation/
14 |           ├── bloc/
15 |               ├── ai_chat_bloc.dart
16 |               ├── ai_chat_event.dart
17 |               └── ai_chat_state.dart
18 |           ├── pages/
19 |               └── ai_chat_page.dart
20 |           └── widgets/
21 |               ├── chat_bubble.dart
22 |               ├── message_input.dart
23 |               └── action_buttons.dart

```

8.6.2 核心代码示例

代码块

```

1  // ai_chat_bloc.dart
2  class AIChatBloc extends Bloc<AIChatEvent, AIChatState> {
3    final AIChatRepository repository;
4    String? sessionId;
5    AIChatBloc({required this.repository}) : super(AIChatInitial()) {
6      on<SendMessage>(_onSendMessage);
7      on<ExecuteAction>(_onExecuteAction);
8    }
9    Future<void> _onSendMessage(SendMessage event, Emitter<AIChatState> emit)
10   async {
11     final currentMessages = state.messages;
12     // 添加用户消息
13     final userMessage = Message(role: 'user', content: event.content);
14     emit(AIChatLoaded(messages: [...currentMessages, userMessage], isLoading:
15     true));
16     try {
17       final response = await repository.chat(
18         sessionId: sessionId,
19         message: event.content,

```

```

18     );
19     sessionId = response.sessionId;
20     // 添加AI回复
21     final aiMessage = Message(
22       role: 'assistant',
23       content: response.reply,
24       structuredData: response.structuredData,
25       suggestedActions: response.suggestedActions,
26     );
27     emit(AIChatLoaded(
28       messages: [...currentMessages, userMessage, aiMessage],
29       isLoading: false,
30     ));
31   } catch (e) {
32     emit(AIChatError(message: '发送失败, 请重试'));
33   }
34 }
35 }
36 // ai_chat_page.dart
37 class AIChatPage extends StatelessWidget {
38   @override
39   Widget build(BuildContext context) {
40     return BlocProvider(
41       create: (context) => AIChatBloc(repository: context.read()),
42       child: Scaffold(
43         appBar: AppBar(
44           title: Row(
45             children: [
46               CircleAvatar(child: Text('🤖')),
47               SizedBox(width: 8),
48               Text('河狸云 AI 助手'),
49             ],
50           ),
51         ),
52         body: Column(
53           children: [
54             Expanded(child: MessageListView()),
55             MessageInputWidget(),
56           ],
57         ),
58       ),
59     );
60   }
61 }

```

8.7 大模型集成方案

8.7.1 推荐方案

方案	优点	缺点	适用场景
----	----	----	------

- | 通义千问 API | 国内访问快、合规 | 需要付费 | 生产环境首选 |
- | 百度文心一言 | 国内合规、稳定 | 需要付费 | 备选方案 |
- | 本地部署 ChatGLM | 数据不出境、可控 | 需要GPU服务器 | 对数据安全要求高 |
- | 混合方案 | 兼顾成本和效果 | 架构复杂 | 推荐 |

8.7.2 混合方案设计

代码块

```
1  @Service
2  public class IntentRecognitionService {
3      @Autowired
4      private RuleBasedRecognizer ruleRecognizer; // 规则引擎
5      @Autowired
6      private LLMRecognizer llmRecognizer; // 大模型
7      public IntentResult recognize(String message, ConversationContext context)
8      {
9          // 1. 先用规则引擎（快、免费、准确率高的场景）
10         IntentResult ruleResult = ruleRecognizer.recognize(message);
11         if (ruleResult != null && ruleResult.getConfidence() > 0.9) {
12             return ruleResult;
13         }
14         // 2. 规则无法判断时，调用大模型
15         return llmRecognizer.recognize(message, context);
16     }
17     // 规则引擎示例
18     @Component
19     public class RuleBasedRecognizer {
20         private static final Map<Pattern, IntentType> PATTERNS = Map.of(
21             Pattern.compile(".*进度.*怎么样.*"), IntentType.QUERY_PROGRESS,
22             Pattern.compile(".*现场.*几个人.*"), IntentType.QUERY_STAFF,
23             Pattern.compile(".*我要投诉.*"), IntentType.ACTION_COMPLAINT,
24             Pattern.compile(".*申请.*变更.*"), IntentType.ACTION_CHANGE
25             // ... 更多规则
26         );
27         public IntentResult recognize(String message) {
28             for (var entry : PATTERNS.entrySet()) {
29                 if (entry.getKey().matcher(message).matches()) {
30                     return IntentResult.of(entry.getValue(), 0.95);
31                 }
32             }
33             return null;
34         }
35     }
36 }
```

```
31         }
32     }
33     return null;
34 }
35 }
```

8.8 参考资料

- [Palantir AIP 架构设计理念](#)
- [Spring AI 官方文档](#)
- [通义千问 API 文档](#)
- [LangChain4j \(Java版LangChain\)](#)

文档维护

版本	日期	修改人	修改内容
V1.0	2026-02	-	初始版本